

Block Array

块状数组

建立块状数组

块状数组，即把一个数组分为几个块，块内信息整体保存，若查询时遇到两边不完整的块直接暴力查询。一般情况下，块的长度为 $\sqrt{n}$ 。详细分析可以阅读 2017 年国家集训队论文中徐明宽的《非常规大小分块算法初探》。

下面直接给出一种建立块状数组的代码。

▼ 实现

```
1 num = sqrt(n);
2 for (int i = 1; i <= num; i++)
3     st[i] = n / num * (i - 1) + 1, ed[i] = n / num * i;
4 ed[num] = n;
5 for (int i = 1; i <= num; i++) {
6     for (int j = st[i]; j <= ed[i]; j++) {
7         belong[j] = i;
8     }
9     size[i] = ed[i] - st[i] + 1;
10 }
```

其中 $st[i]$ 和 $ed[i]$ 为块的起点和终点， $size[i]$ 为块的大小。

保存与修改块内信息

例题 1: [教主的魔法](#)

两种操作：

1. 区间 $[l, r]$ 每个数都加上 x ；
2. 查询区间 $[l, r]$ 内大于等于 x 的数的个数。

我们要询问一个块内大于等于一个数的数的个数，所以需要有一个 t 数组对块内排序， a 为原来的（未被排序的）数组。对于整块的修改，使用类似于标记永久化的方式，用 $delta$ 数组记录现在块内整体加上的值。设 q 为查询和修改的操作次数总和，则时间复杂度 $O(\sqrt{n}q)$ 。

用 $delta$ 数组记录每个块的整体赋值情况。

▼ 实现

```

1 void Sort(int k) {
2     for (int i = st[k]; i <= ed[k]; i++) t[i] = a[i];
3     sort(t + st[k], t + ed[k] + 1);
4 }
5
6 void Modify(int l, int r, int c) {
7     int x = belong[l], y = belong[r];
8     if (x == y) // 区间在一个块内就直接修改
9     {
10         for (int i = l; i <= r; i++) a[i] += c;
11         Sort(x);
12         return;
13     }
14     for (int i = l; i <= ed[x]; i++) a[i] += c; // 直接修改起始段
15     for (int i = st[y]; i <= r; i++) a[i] += c; // 直接修改结束段
16     for (int i = x + 1; i < y; i++) delta[i] += c; // 中间的块整体打上标记
17     Sort(x);
18     Sort(y);
19 }
20
21 int Answer(int l, int r, int c) {
22     int ans = 0, x = belong[l], y = belong[r];
23     if (x == y) {
24         for (int i = l; i <= r; i++)
25             if (a[i] + delta[x] >= c) ans++;
26         return ans;
27     }
28     for (int i = l; i <= ed[x]; i++)
29         if (a[i] + delta[x] >= c) ans++;
30     for (int i = st[y]; i <= r; i++)
31         if (a[i] + delta[y] >= c) ans++;
32     for (int i = x + 1; i <= y - 1; i++)
33         ans +=
34             ed[i] - (lower_bound(t + st[i], t + ed[i] + 1, c - delta[i]) - t) + 1;
35     // 用 lower_bound 找出中间每一个整块中第一个大于等于 c 的数的位置
36     return ans;
37 }

```

例题 2：寒夜方舟

两种操作：

1. 区间 每个数都变成；
2. 查询区间 内小于等于 的数的个数。

用 `delta` 数组记录现在块内被整体赋值为何值。当该块未被整体赋值时，用一个特殊值（如 `0x3f3f3f3f3f3f3f11`）加以表示。对于边角块，查询前要 `pushdown`，把块内存的信息下放到每一个数上。赋值之后记得重新 `sort` 一遍。其他方面同上题。

▼ 实现

```

1 void Sort(int k) {
2     for (int i = st[k]; i <= ed[k]; i++) t[i] = a[i];
3     sort(t + st[k], t + ed[k] + 1);
4 }
5
6 void PushDown(int x) {
7     if (delta[x] != 0x3f3f3f3f3f3f3f11) // 用该值标记块内没有被整体赋值
8         for (int i = st[x]; i <= ed[x]; i++) a[i] = t[i] = delta[x];
9     delta[x] = 0x3f3f3f3f3f3f3f11;
10 }
11
12 void Modify(int l, int r, int c) {
13     int x = belong[l], y = belong[r];
14     PushDown(x);
15     if (x == y) {
16         for (int i = l; i <= r; i++) a[i] = c;
17         Sort(x);
18         return;
19     }
20     PushDown(y);
21     for (int i = l; i <= ed[x]; i++) a[i] = c;
22     for (int i = st[y]; i <= r; i++) a[i] = c;
23     Sort(x);
24     Sort(y);
25     for (int i = x + 1; i < y; i++) delta[i] = c;
26 }
27
28 int Binary_Search(int l, int r, int c) {
29     int ans = l - 1, mid;
30     while (l <= r) {
31         mid = (l + r) / 2;
32         if (t[mid] <= c)
33             ans = mid, l = mid + 1;
34         else
35             r = mid - 1;
36     }
37     return ans;
38 }
39
40 int Answer(int l, int r, int c) {
41     int ans = 0, x = belong[l], y = belong[r];
42     PushDown(x);
43     if (x == y) {
44         for (int i = l; i <= r; i++)
45             if (a[i] <= c) ans++;
46         return ans;
47     }
48     PushDown(y);
49     for (int i = l; i <= ed[x]; i++)
50         if (a[i] <= c) ans++;
51     for (int i = st[y]; i <= r; i++)
52         if (a[i] <= c) ans++;
53     for (int i = x + 1; i <= y - 1; i++) {
54         if (0x3f3f3f3f3f3f3f11 == delta[i])
55             ans += Binary_Search(st[i], ed[i], c) - st[i] + 1;
56         else if (delta[i] <= c)
57             ans += size[i];
58     }
59     return ans;
60 }

```

练习

1. [单点修改, 区间查询](#)
 2. [区间修改, 区间查询](#)
 3. [【模板】线段树 2](#)
 4. [「Ynoi2019 模拟赛」Yuno loves sqrt technology III](#)
 5. [「Violet」蒲公英](#)
 6. [作诗](#)
-

本页面最近更新: 2023/2/18 07:57:07, [更新历史](#)

发现错误? 想一起完善? [在 GitHub 上编辑此页!](#)

本页面贡献者: [Alpacabla](#), [chenyanlann](#), [Enter-tainer](#), [Henry-ZHR](#), [iamtwz](#), [lr1d](#), [ksyx](#), [Marcythm](#), [ouuan](#), [partychicken](#), [R-G-Mocoratioen](#), [zhuzeyu22](#)

本页面的全部内容 in [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供, 附加条款亦可能应用